FIELD NOTES

LLM API Field Guide

An LLM API bills you whether or not the answer was any good, and most of the ways it goes wrong do not raise an exception. Every script here is read only.

97 fixes, each one a written note with diagrams and a small Python and Node.js script that finds the problem and repairs it. All 97 have a script in the public repo.

Before you run anything. Every script starts in a dry run: it reports what it would change and writes nothing. Read that output first, then set `DRY_RUN=false` when you are satisfied it has understood your data.

Scripts: github.com/allanninal/llm-api-fixes
Guides: allanninal.dev/llm
All 14 repos: github.com/allanninal

The fixes

1 **429s while every minute sits under the configured limit**

The backfill went out at 09:00 and 429ed for eleven minutes. Somebody pulled the usage report afterwards, and the worst minute in the whole window used a fifth of the organization's input tokens per minute. Nobody believes the graph, so the ticket says rate limit increase required and the increase, when it arrives, changes nothing: the same job trips at the same point next Tuesday, still using a fifth of a limit that is now twice as big.

`anthropic_ramp_acceleration.py`

https://www.allanninal.dev/llm/acceleration-limit-on-traffic-spike/

https://github.com/allanninal/llm-api-fixes/tree/main/acceleration-limit-on-traffic-spike

---

2 **anthropic-version is missing, ancient, or added in transit**

The webhook receiver has worked for a year. It is forty lines of fetch written the afternoon somebody needed it, it posts to Claude, and it has never once been touched. Today it returns 400 invalid_request_error on every call and the message is about a header you have never had to think about, because the SDK sets it and this thing is not the SDK. The part worth understanding is not the fix, which is one line. It is why it worked yesterday: the request used to go through the gateway, the gateway added anthropic-version for you, and last week somebody pointed that service straight at the API.

`anthropic_version_header_probe.py`

https://www.allanninal.dev/llm/anthropic-version-header-missing-or-ancient/

https://github.com/allanninal/llm-api-fixes/tree/main/anthropic-version-header-missing-or-ancient

---

3 **API keys that no request has ever used**

Nobody left. Nobody was removed from anything. The engineer who minted the key in March is sitting eight feet away, still employed, still on the project, and could tell you within a second why she made it: a vendor evaluation that went nowhere. The key has authenticated exactly zero requests in the five months since, and it has full access to everything in that project, and there is no report anywhere that will ever mention it, because a credential with no traffic produces no cost line, no log entry and no alert. Creating it took one click. Deleting it requires somebody to be confident nothing breaks.

`llm_idle_key_audit.py`

https://www.allanninal.dev/llm/api-key-never-used/

https://github.com/allanninal/llm-api-fixes/tree/main/api-key-never-used

---

4 **an archived project still holds live API keys**

The prototype was shut down in the spring. The project was archived, which felt like closing it: it vanished from the console's project switcher and from every list anyone looks at. Nothing inside it was touched. Two keys are still enabled, one of them authenticated a request last Tuesday, and the quarterly key audit has never seen either of them, because the audit iterates projects and archived projects are not in the default listing.

`openai_archived_project_keys.py`

https://www.allanninal.dev/llm/archived-project-still-holds-keys/

https://github.com/allanninal/llm-api-fixes/tree/main/archived-project-still-holds-keys

---

5 **The Assistants API is shut down. Is yours still answering?**

The pipeline has not run since Wednesday and the error is a 404 on a path that has been in the codebase for two years. Somebody checks the model id first, because that is what a 404 usually means, and the model id is fine. It is fine because the model was never the problem: the whole /v1/assistants family reached its published shutdown date on 26 August 2026 and the endpoint is simply not there any more. The uncomfortable part comes an hour later, when the same probe run against the staging organization returns 200, and now you have two organizations, one dead API, and a question nobody wants to answer about how long the other one has.

`assistants_shutdown_probe.py`

https://www.allanninal.dev/llm/assistants-api-already-shut-down/

https://github.com/allanninal/llm-api-fixes/tree/main/assistants-api-already-shut-down

---

6 **audio and image usage never shows up in a token dashboard**

The internal dashboard has been within a few percent of the invoice for a year, and a few percent is what everyone expects a dashboard to be. It is not rounding. It is the text-to-speech in the mobile app, the transcription on the support calls, the thumbnails the marketing tool generates, and the web search the agent does before it answers. None of those are denominated in tokens, and the endpoint the dashboard was built on only knows about tokens.

`openai_modality_spend_reconcile.py`

https://www.allanninal.dev/llm/audio-and-image-line-items-unnoticed/

https://github.com/allanninal/llm-api-fixes/tree/main/audio-and-image-line-items-unnoticed

---

7 **The background response is queued and nothing is polling**

The queue worker takes a job, starts a background response, writes the id somewhere, and returns. It is a good design: the request is accepted in a few hundred milliseconds and the model can take twenty minutes without holding a socket open. Then the worker is redeployed mid-shift, or the process that was going to poll gets an unhandled exception on a different code path, and the ids stop being read. The jobs keep running. They keep billing. And the only place their results exist is on an object nobody is asking about.

`openai_background_response_audit.py`

https://www.allanninal.dev/llm/background-response-never-polled/

https://github.com/allanninal/llm-api-fixes/tree/main/background-response-never-polled

---

8 **Cancelling a batch does not unbill the rows it already ran**

Somebody hit cancel during the deploy, which was the right call, and then the runbook said re-run it in the morning. So it was re-run in the morning. What nobody checked is that the batch had already processed sixty-one thousand of its ninety thousand rows before the cancel landed, that those rows are sitting in the output file, and that the morning re-run paid for all sixty-one thousand of them a second time. Cancel is not a rollback. It is a stop.

`batch_cancellation_audit.py`

https://www.allanninal.dev/llm/batch-cancelled-partial-results/

https://github.com/allanninal/llm-api-fixes/tree/main/batch-cancelled-partial-results

---

9 **scheduled jobs pay full price for work the Batch API halves**

Nothing here is broken. No request failed, no row is missing, and no alert should have fired. The nightly enrichment job fires forty thousand completions between 02:00 and 02:20, finishes cleanly, and does it again the next night. It has no user waiting on it and no latency requirement of any kind, and it is being billed at the interactive rate because the synchronous endpoint is what the SDK example used. This is not a bug report. It is an invoice roughly twice the size it needs to be.

`openai_batch_discount_audit.py`

https://www.allanninal.dev/llm/batch-discount-left-unused/

https://github.com/allanninal/llm-api-fixes/tree/main/batch-discount-left-unused

---

10 **the batch left an error_file_id that nothing ever fetched**

The Batch API answers in two files. Successes go to output_file_id and failures go to error_file_id, and the ingest code was written against the first one on a day when the test batch had no failures. It has never opened the second. The failures are not lost — they were written down carefully, one JSON line each, with the custom_id and the reason. They are sitting in a file that has an id, a byte count and an expiry date, and in thirty days they will be gone whether or not anybody looked.

`openai_batch_error_file_audit.py`

https://www.allanninal.dev/llm/batch-error-file-never-read/

https://github.com/allanninal/llm-api-fixes/tree/main/batch-error-file-never-read

---

11 **a batch expired when the 24 hour completion window closed**

The submission returned 200 yesterday afternoon. The poller has been asking for the batch ever since, testing the status against completed, and it has never matched, so as far as the job is concerned the work is still running. It is not. Twenty-four hours after the batch started processing, everything OpenAI had not got to was abandoned, the status went to expired, and thirty thousand rows landed in the error file with the code batch_expired. Nothing raised, nothing retried, and the poller is still waiting.

`openai_batch_expiry_audit.py`

https://www.allanninal.dev/llm/batch-expired-past-24h-window/

https://github.com/allanninal/llm-api-fixes/tree/main/batch-expired-past-24h-window

---

12 **The batch failed validation and named the broken line**

The submitter returned 200 and the run log says the nightly enrichment fired. Sixteen hours later the enrichment table is exactly as long as it was yesterday. There was no exception, no alert and no retry, because from the client's point of view nothing went wrong: the batch was accepted. It failed forty seconds later, in a state your code never looks at, and it has been holding a list of line numbers ever since.

`openai_batch_validation_audit.py`

https://www.allanninal.dev/llm/batch-failed-input-validation/

https://github.com/allanninal/llm-api-fixes/tree/main/batch-failed-input-validation

---

13 **The batch finished and nobody ever collected the output**

The batch ran, the model answered ninety thousand times, the invoice includes every one of those answers, and the file holding them was deleted a month later without ever being opened. There is no incident for this. The batch object still sits in the list saying completed, which is true, and pointing at an id that no longer resolves, which is also true. The only party that ever knew the results were wanted was a process that stopped running in March.

`batch_output_unclaimed_audit.py`

https://www.allanninal.dev/llm/batch-output-file-never-downloaded/

https://github.com/allanninal/llm-api-fixes/tree/main/batch-output-file-never-downloaded

---

14 **a batch reads completed while some of its rows failed**

The nightly job submits fifty thousand rows, sleeps, polls until the status turns to completed, downloads the output file and loads it. It has done that every night for eight months. The table it fills is short — not empty, not obviously wrong, just a few hundred rows smaller than the input file, by a number that changes every night. No exception was ever raised. The batch object says completed in plain text, and three fields further down it says "failed": 869, which nothing has ever read.

`openai_batch_partial_failure_audit.py`

https://www.allanninal.dev/llm/batch-partial-failure-unnoticed/

https://github.com/allanninal/llm-api-fixes/tree/main/batch-partial-failure-unnoticed

---

15 **The batch queue is full, so the next submission is refused**

Nothing has failed. Every batch in the account is healthy, every one of them is inside its window, and the Messages API limits are not being touched. The only symptom is that the submitter has started getting 429s, and it is getting them for a reason that has nothing to do with the requests it is sending: an unrelated job, in an unrelated workspace, has parked four hundred thousand requests in a queue that the whole organization shares.

`anthropic_batch_queue_depth.py`

https://www.allanninal.dev/llm/batch-queue-limit-reached/

https://github.com/allanninal/llm-api-fixes/tree/main/batch-queue-limit-reached

---

16 **Cache read share stepped down the day the model changed**

The migration was a one-line change and it went perfectly: latency improved, quality held, the per-token rate went down. Three weeks later the input bill was up by a third and nobody could see why, because the thing that changed is not on the invoice as a line item. The cache-read share was sixty-eight per cent for a fortnight, dropped to eleven on the day the new model id first appears in the usage report, and has been eleven ever since. The prompt was never edited.

`anthropic_cache_step_after_model_switch.py`

https://www.allanninal.dev/llm/cache-hit-rate-collapsed-after-model-change/

https://github.com/allanninal/llm-api-fixes/tree/main/cache-hit-rate-collapsed-after-model-change

---

17 **Cache written on every call by a prefix that keeps moving**

Caching went in six weeks ago and the cached share never moved off zero. The obvious explanations were checked and cleared: the breakpoint is there, the prefix is long, the traffic is constant. What nobody looked at until somebody pulled the minute buckets is that the writes are not occasional. There is a write in every single minute of the window, one after another for four hours, and not one read anywhere in them. A five minute entry written at 14:03 was still alive at 14:07, and the call at 14:07 wrote a new one.

`anthropic_cache_prefix_churn.py`

https://www.allanninal.dev/llm/cache-invalidated-by-changing-prefix/

https://github.com/allanninal/llm-api-fixes/tree/main/cache-invalidated-by-changing-prefix

---

18 **cache writes are paid for and never read back**

Caching was switched on in June and the bill went up. Every call writes a fresh cache entry, is billed 1.25x base input for the privilege, and then nothing ever reads that entry back before it expires. The reason is one line: a request id got templated into the system prompt, ahead of the breakpoint, so no two prefixes have ever been byte-identical. The feature is working exactly as documented and it is costing you money.

`anthropic_cache_write_ratio.py`

https://www.allanninal.dev/llm/cache-writes-with-no-reads/

https://github.com/allanninal/llm-api-fixes/tree/main/cache-writes-with-no-reads

---

19 **Claude Code edits rejected more often than they are kept**

The diff comes up, it is forty lines, it is confidently wrong about where the validation lives, and it gets rejected in about two seconds. That happens eleven times before lunch, and none of those eleven rejections is recorded anywhere a person would look, because rejecting a proposal is not an error and not a failure and not, from the tool's point of view, an event worth complaining about. Every one of those forty-line diffs was generated at frontier output rates and paid for in full before anybody read it.

`claude_code_edit_acceptance.py`

https://www.allanninal.dev/llm/claude-code-edit-rejection-rate-high/

https://github.com/allanninal/llm-api-fixes/tree/main/claude-code-edit-rejection-rate-high

---

20 **Claude Code sessions billed with zero cache reads**

Two developers on the same team, the same repository, the same model, and one of them costs four times what the other does. The expensive one is not doing more work — fewer commits, in fact — and the cheap one has not configured anything. The difference is a habit: one of them keeps a session open and talks to it, and the other opens a fresh session for every question, which feels tidier and means the project context, the tool definitions and the file contents are paid for again at full rate every single time.

`claude_code_cache_coverage.py`

https://www.allanninal.dev/llm/claude-code-sessions-not-hitting-cache/

https://github.com/allanninal/llm-api-fixes/tree/main/claude-code-sessions-not-hitting-cache

---

21 **code execution has spent its free 1,550 container hours**

The analytics route attaches the customer's CSV to the request, because sometimes the question is about the numbers in it and when it is, the model should be able to run something. Sometimes is about one call in forty. The other thirty-nine attach the file, spin up a container to hold it, ask something that needs no arithmetic at all, and are billed for five minutes of a machine that was never asked to do anything.

`anthropic_code_execution_hours_audit.py`

https://www.allanninal.dev/llm/code-execution-hours-exceed-free-allowance/

https://github.com/allanninal/llm-api-fixes/tree/main/code-execution-hours-exceed-free-allowance

---

22 **Cost lands in the default workspace and cannot be charged back**

The chargeback spreadsheet has one row that never shrinks. Four workspaces are named after the teams that own them and add up to about sixty per cent of the bill; the rest arrives under a heading somebody typed once as Unallocated and has been carrying forward every month since. It is not a rounding error and it is not fraud. It is the default workspace, and it is reported as null, which is not a workspace you can go and talk to.

`anthropic_default_workspace_cost.py`

https://www.allanninal.dev/llm/default-workspace-cost-unattributable/

https://github.com/allanninal/llm-api-fixes/tree/main/default-workspace-cost-unattributable

---

23 **An empty vector store is still named in vector_store_ids**

The retrieval feature is live and nobody has complained, which is the part that should worry you. Every request carries a file_search tool with a vector_store_ids array copied out of the config months ago; every response comes back 200 with no citations attached; and the model, asked what the refund window is, says thirty days in a confident and well-structured paragraph. It is thirty days in the training data. Your policy changed to fourteen in March, it is in the document you indexed, and the store that was supposed to hold that document has nothing in it.

`openai_empty_vector_store_audit.py`

https://www.allanninal.dev/llm/empty-vector-store-still-referenced/

https://github.com/allanninal/llm-api-fixes/tree/main/empty-vector-store-still-referenced

---

24 **An expired file still answers metadata and fails every use**

The document pipeline has been fine for months and this morning a batch of requests started failing on a handful of customers. Not all of them, and not consistently. The error is a 404, which sends everybody looking for a typo, and the id is right — you can paste it into a metadata call and get a filename, a size and a created date back. The file exists. It answers. And every request that tries to actually use it fails before inference, because what came back was the label and the thing behind it went away eleven days ago.

`anthropic_expired_file_refs.py`

https://www.allanninal.dev/llm/expired-files-still-referenced/

https://github.com/allanninal/llm-api-fixes/tree/main/expired-files-still-referenced

---

25 **A CMEK key config is inert but assumed to be encrypting**

The answer in the security questionnaire is a single sentence and it took three weeks of work to be able to write it: customer data is encrypted at rest under a key we control, in our own KMS, which we can revoke. The engineer who did that work created the key, registered it, and moved on to the next ticket, because from the outside everything looked done — the config was there, it had the right ARN, the console showed it. What nobody did was the second step, and there is no error anywhere that says so, because an encryption config that is not attached to anything does not fail. It just is not used.

`anthropic_cmek_external_key_audit.py`

https://www.allanninal.dev/llm/external-key-config-unattached/

https://github.com/allanninal/llm-api-fixes/tree/main/external-key-config-unattached

---

26 **fast mode billed at twice the rate and served as default**

Somebody turned on Fast mode for the checkout assistant eight months ago, in the console, in an afternoon nobody wrote down. The latency graph looked better for a week. It does not look better now, and the team has spent two sprints on the retrieval step trying to work out why. The requests still carry the premium tier, the responses still return 200, and the field that says which tier actually served them is not the field anyone is logging.

`openai_fast_mode_tier_audit.py`

https://www.allanninal.dev/llm/fast-mode-silently-downgraded/

https://github.com/allanninal/llm-api-fixes/tree/main/fast-mode-silently-downgraded

---

27 **Files pile up against a storage ceiling nothing reports**

The batch pipeline has run every night for two years and tonight it stops on the first step, uploading the input file, with an error nobody has seen before. Nothing was deployed. The key works, because the same key just listed a thousand files without complaint. Reads are fine. Retrieval is fine. Inference is fine. The only thing that is broken is writing, and it is broken because a number you have never once looked at reached a ceiling you have never once been shown.

`file_store_quota_audit.py`

https://www.allanninal.dev/llm/files-accumulating-against-storage-quota/

https://github.com/allanninal/llm-api-fixes/tree/main/files-accumulating-against-storage-quota

---

28 **The fine-tuning job failed and error.code was never read**

Someone kicked off the training run on a Thursday afternoon, watched the 200 come back, pasted the job id into the channel, and went home. The following Tuesday the deploy that was supposed to point at the new model is still pointing at the old one, which nobody notices because it works. Three weeks later a stakeholder asks how the fine-tune is performing and the honest answer turns out to be that it never trained. The job object has been sitting there the entire time with a status, an error code and the name of the field that was wrong, and nothing ever asked it.

`openai_fine_tune_failures.py`

https://www.allanninal.dev/llm/fine-tune-job-failed-with-error-code/

https://github.com/allanninal/llm-api-fixes/tree/main/fine-tune-job-failed-with-error-code

---

29 **a fine-tuned model was trained, billed, and never called once**

There was a quarter when fine-tuning was going to be the answer. Four jobs were queued over three weeks, each on a slightly better training file, and the last one came back with a loss curve somebody screenshotted into Slack. Then the base model got better, or the prompt got better, or the person who cared moved teams. The model ids are still there. They still resolve. They have between them served zero requests, and the training was invoiced the month it ran.

`openai_fine_tune_usage_audit.py`

https://www.allanninal.dev/llm/fine-tuned-model-never-used/

https://github.com/allanninal/llm-api-fixes/tree/main/fine-tuned-model-never-used

---

30 **Fine-tuning stops taking new jobs while old ones keep serving**

Nothing is broken, which is the problem. The classifier fine-tune is serving traffic, the job list is full of green, and the plan — retrain it in the new year when there is time — sounds entirely reasonable to everybody in the room. It is not reasonable, and the reason is not on any dashboard: nobody has run inference against a fine-tuned model in about nine weeks, because the classifier was quietly replaced by a prompt in June and only the old batch job still calls it. The right to create a fine-tuning job expired somewhere in there. There was no notification, because nothing happened.

`fine_tuning_gate_audit.py`

https://www.allanninal.dev/llm/fine-tuning-jobs-blocked/

https://github.com/allanninal/llm-api-fixes/tree/main/fine-tuning-jobs-blocked

---

31 **The flex tier fails by not being served, and bills nothing**

The nightly enrichment job was moved to flex processing because it is not urgent and flex is priced like batch. It has been fine for six weeks. It is still fine, in the sense that nothing has ever paged: the job logs a count at the end, the count is lower some nights, and the difference is a few thousand records that quietly did not get enriched. The invoice is lower too, which is exactly what everybody expected to see, and which is why nobody looked.

`openai_flex_tier_served.py`

https://www.allanninal.dev/llm/flex-resource-unavailable-timeouts/

https://github.com/allanninal/llm-api-fixes/tree/main/flex-resource-unavailable-timeouts

---

32 **a floating model alias silently changes model under you**

No error, no deploy, no incident. The evals were 91% on Thursday and 87% on Monday, the mean output length moved, the prompt cache hit rate dropped a few points and the bill went up slightly. Everyone looks at the prompt, which did not change, and at the retrieval corpus, which did not change either. The model changed: the config names an alias, and an alias is a pointer, not a model.

`anthropic_alias_pinning_audit.py`

https://www.allanninal.dev/llm/floating-alias-instead-of-pinned-snapshot/

https://github.com/allanninal/llm-api-fixes/tree/main/floating-alias-instead-of-pinned-snapshot

---

33 **a frontier model is answering twenty-token questions**

Somebody built the intent router in an afternoon eighteen months ago. They pasted the model name out of the quickstart, because on that afternoon the question was whether the thing worked at all and the answer was worth whatever it cost. It works. It has worked every day since, four hundred thousand times a month, and every one of those calls returns a single word from a list of nine. The model that returns it is the most expensive one the organization can buy.

`openai_model_rightsizing_audit.py`

https://www.allanninal.dev/llm/frontier-model-on-trivial-workload/

https://github.com/allanninal/llm-api-fixes/tree/main/frontier-model-on-trivial-workload

---

34 **The anthropic-beta value that 400s, and the one that went GA**

The pull request adds one header and it is nine characters different from the one in the documentation. Every request the service makes now returns 400, the message names the header, and it takes about four minutes to spot. That is the easy half. The hard half is the header three services along that is spelled perfectly, was correct when it was written, returns 200 today, and is quietly holding that client on a response shape the platform stopped documenting in August — no error, no warning, and a list endpoint that has been missing expires_at for months.

`anthropic_beta_header_audit.py`

https://www.allanninal.dev/llm/invalid-beta-header-value/

https://github.com/allanninal/llm-api-fixes/tree/main/invalid-beta-header-value

---

35 **ITPM runs out because uncached input is never cached**

The 429s arrive in the afternoon and the team does what teams do: fewer workers, longer sleeps, a queue in front. None of it moves. The request counter was never the thing that ran out. What ran out was input tokens per minute, and the reason is that the same forty thousand tokens of system prompt and tool schemas are sent uncached on every single call, and every one of those tokens is charged against the limiter.

`anthropic_itpm_headroom.py`

https://www.allanninal.dev/llm/itpm-exhausted-uncached-input/

https://github.com/allanninal/llm-api-fixes/tree/main/itpm-exhausted-uncached-input

---

36 **keys still work after their owner loses project access**

She left in March. The laptop came back, SSO was switched off the same afternoon, and the offboarding ticket was closed with every box ticked. The API key she minted in her second week is still in the environment of a nightly job, still authenticating, still billing. Nothing revoked it, because nothing was ever asked to: the key is a separate object from her membership, and removing the membership left the key exactly as it was.

`openai_orphaned_key_audit.py`

https://www.allanninal.dev/llm/key-owner-lost-project-access/

https://github.com/allanninal/llm-api-fixes/tree/main/key-owner-lost-project-access

---

37 **Production keys owned by people, not service accounts**

The service has been up for two years and it has never once paged anybody about credentials. That is because the credential is Marco's. He minted it in the first week, before the project structure existed, because a personal key works the instant you create it and a service account is a thing you have to think about first. Nothing has gone wrong. Marco is still here, still on the team, still the person who would fix it. The only fact that has changed in two years is that eleven thousand dollars a month now moves through a credential whose lifecycle is attached to one person's employment rather than to the service's.

`openai_user_owned_key_audit.py`

https://www.allanninal.dev/llm/legacy-user-owned-keys-in-project/

https://github.com/allanninal/llm-api-fixes/tree/main/legacy-user-owned-keys-in-project

---

38 **A live project's usage buckets have been empty for days**

A customer asks, politely, when the summaries are coming back. Nobody on the call knows what they mean, because the feature works: the page renders, the job runs, the queue is empty, the error rate is zero and the latency graph is the flattest it has been all year. It is flat because for eleven days that project has sent the API nothing at all. A feature flag went the wrong way in an unrelated release, and every dashboard the team owns measures things that only exist when requests do.

`openai_project_went_quiet.py`

https://www.allanninal.dev/llm/live-project-zero-usage-buckets/

https://github.com/allanninal/llm-api-fixes/tree/main/live-project-zero-usage-buckets

---

39 **The 1M context window is capped at 200k in your own code**

The long-context work took a quarter. There is a constant called MAX_CONTEXT_TOKENS, there is a guard that truncates the retrieved documents before they reach it, there is a branch that routes anything over the line to a summarise-first path, and there is a comment above all of it citing a beta header. Every one of those was correct when it was written. The model ids in the config have changed four times since, the window they now report is a million tokens, and the constant still says two hundred thousand, so the truncation still fires, the summarise path still runs, and the capability the company is paying for stops at the same place it stopped eighteen months ago.

`anthropic_context_window_cap.py`

https://www.allanninal.dev/llm/long-context-gated-on-obsolete-beta/

https://github.com/allanninal/llm-api-fixes/tree/main/long-context-gated-on-obsolete-beta

---

40 **most of your input tokens sit in the 200k-1M band**

The agent keeps everything. That was the design: give it the whole ticket history, the whole document, every tool result it has ever produced in this session, and let it decide what matters. It worked beautifully in testing, where a session was four turns. In production a session is forty, each turn carries everything the previous thirty-nine produced, and the prefix has quietly grown to four hundred thousand tokens that get sent again from scratch every single time somebody types.

`anthropic_long_context_audit.py`

https://www.allanninal.dev/llm/long-context-requests-unwatched/

https://github.com/allanninal/llm-api-fixes/tree/main/long-context-requests-unwatched

---

41 **max_tokens is set above the model's own output cap**

The classifier was moved onto the cheap model on a Friday, which is the correct thing to do with a classifier. It is the same helper function everything else uses, so it inherited the same max_tokens, which is a number somebody picked for the model that writes reports. Every call to it now comes back 400, and the message says exactly what is wrong, and the message is in a log that no dashboard reads.

`anthropic_max_tokens_cap.py`

https://www.allanninal.dev/llm/max-tokens-above-model-cap/

https://github.com/allanninal/llm-api-fixes/tree/main/max-tokens-above-model-cap

---

42 **a model id in use is past its published shutdown date**

Nothing was deployed. The key did not rotate. One model id started returning 404 on the same morning for every request, and the message reads exactly like a typo: The model does not exist or you do not have access to it. It existed yesterday. There is no distinct error code for a retired model, no deprecation warning on the successful calls that came before it, and nothing in the response that tells the difference between a model that was shut down and a model name somebody misspelled.

`openai_model_shutdown_audit.py`

https://www.allanninal.dev/llm/model-past-shutdown-date/

https://github.com/allanninal/llm-api-fixes/tree/main/model-past-shutdown-date

---

43 **a model you still call retires in under 90 days**

Every call is returning 200. Latency is normal, cost is normal, the evals are green. The only thing wrong is a field nobody is reading: shutdown_date on the model you route most of your traffic to is a real date, and it is close. Nothing will change until that morning, and then everything will, all at once, in every code path that names the id.

`openai_model_retirement_window.py`

https://www.allanninal.dev/llm/model-retiring-within-90-days/

https://github.com/allanninal/llm-api-fixes/tree/main/model-retiring-within-90-days

---

44 **Nothing has ever called the free moderation endpoint**

The product takes free text from anyone with the link and has done since launch. Nobody thinks of it as a moderation problem, because nothing has gone wrong yet and the model refuses most of what it should refuse on its own. Then a support ticket arrives with a screenshot, and somebody asks the only question that matters in the first ten minutes: what do we already screen? The honest answer takes an afternoon to establish, and it turns out to be nothing — not because anyone decided against it, but because moderation is a separate endpoint you have to go and call, and no line of code ever did.

`openai_moderation_coverage_audit.py`

https://www.allanninal.dev/llm/moderation-never-called/

https://github.com/allanninal/llm-api-fixes/tree/main/moderation-never-called

---

45 **no hard spend limit is set, so the bill has no ceiling**

The console shows a budget chart, and everybody who has looked at it believes it is a brake. It is not; it is a chart. The hard limit is a separate object on a separate admin endpoint, it is off by default, and until somebody turns it on there is no amount of spend in a month that will cause a request to be refused.

`openai_spend_limit_audit.py`

https://www.allanninal.dev/llm/no-organization-spend-limit/

https://github.com/allanninal/llm-api-fixes/tree/main/no-organization-spend-limit

---

46 **One project holds every environment, so nothing can be capped**

Somebody asks what production costs. Not the API bill, which everyone knows to the dollar, but the production share of it, and the answer takes four days to arrive because there is no answer. Every request the company has ever made to OpenAI — the customer-facing assistant, the nightly evaluation suite, the CI job that regenerates fixtures, eleven laptops, and the prototype somebody wrote in March and never turned off — landed in a project called Default project that was created before anybody had an opinion about structure.

`openai_project_boundary_audit.py`

https://www.allanninal.dev/llm/no-prod-dev-project-separation/

https://github.com/allanninal/llm-api-fixes/tree/main/no-prod-dev-project-separation

---

47 **A non-streaming request over 10 minutes times out with 504**

The prompt is two thousand tokens. The context window is nowhere near full. Nothing is too large by any measure anybody in the room has checked, and the request still dies — sometimes as 504 with timeout_error, more often as nothing at all, because a load balancer somewhere between you and Anthropic closed an idle connection while the model was still writing. The ceiling this hit is a clock.

`anthropic_wall_clock_preflight.py`

https://www.allanninal.dev/llm/non-streaming-request-over-ten-minutes/

https://github.com/allanninal/llm-api-fixes/tree/main/non-streaming-request-over-ten-minutes

---

48 **one line item or project is most of the organization's bill**

The bill is roughly what everyone expected, which is why nobody has opened it. When somebody finally does, one line item is seventy-eight percent of it, and it is not the one the team spent last quarter optimising. There is no error here and nothing failed. There is a month of engineering time that went into a row worth three percent of the total, because the report was read as a single number and single numbers do not have a shape.

`openai_cost_concentration_audit.py`

https://www.allanninal.dev/llm/one-model-or-project-dominates-cost/

https://github.com/allanninal/llm-api-fixes/tree/main/one-model-or-project-dominates-cost

---

49 **Organization invites sat pending until they expired**

The new engineer said she was set up, and she was: the laptop arrived, SSO worked, the repository cloned. Six weeks later somebody notices that the nightly job is running under a colleague's personal key because she asked to borrow it on her second day and never stopped. The invite to the API organization was sent on her first morning, went to a filtered folder, and is still sitting in the list marked pending with an expires_at that passed in April.

`openai_stale_invite_audit.py`

https://www.allanninal.dev/llm/openai-invites-pending-past-expiry/

https://github.com/allanninal/llm-api-fixes/tree/main/openai-invites-pending-past-expiry

---

50 **Model visible, streaming refused: the org is unverified**

The nightly summarisation job is fine. The evaluation suite is fine. CI is fine. The chat panel in the product returns nothing at all, and has since the release that switched it to the newer model, and the only thing anyone can say about it is that it used to work. The model id is right — you checked, it resolves. The key is right — it is the same key the job uses. The difference between the route that works and the route that does not is one field in the request body, "stream": true, and the reason it fails has nothing to do with your code at all.

`openai_streaming_verification_probe.py`

https://www.allanninal.dev/llm/org-verification-required/

https://github.com/allanninal/llm-api-fixes/tree/main/org-verification-required

---

51 **purpose=assistants files outlived the API that owned them**

The Assistants API reached its shutdown date on 26 August 2026 and the migration was done months ago. Runs became responses, threads became conversations, the beta header came out of the client, and the whole thing has been working ever since. What nobody did, because nothing asked and nothing broke, was go and look at the files. They are still there. Every document ever uploaded for an assistant, every code-interpreter output from a run that no longer exists, sitting in a store whose owner was deleted, counting against a ceiling and answering every listing call as though the last two years never happened.

`openai_orphaned_assistant_files.py`

https://www.allanninal.dev/llm/orphaned-assistants-purpose-files/

https://github.com/allanninal/llm-api-fixes/tree/main/orphaned-assistants-purpose-files

---

52 **output tokens per minute is the real ceiling, not RPM**

The capacity plan is a spreadsheet with requests per minute in it, and it has been right about everything for a year. Then thinking gets turned up on the summariser, and the same number of calls starts 429ing. Nobody changed the request rate. Nobody changed the prompts. What changed is that each answer got four times longer, and the limiter that was never in the spreadsheet is the one counting characters as they come out.

`anthropic_otpm_ceiling.py`

https://www.allanninal.dev/llm/otpm-exhausted/

https://github.com/allanninal/llm-api-fixes/tree/main/otpm-exhausted

---

53 **output tokens, not input, are what the bill is made of**

Every conversation about LLM cost starts with the prompt. The system prompt gets trimmed, the few-shot examples get cut, someone measures the context window. Then you group the cost report by token_type and find that three-quarters of the money is on the other side of the request, where none of the levers you just pulled reach.

`anthropic_output_cost_audit.py`

https://www.allanninal.dev/llm/output-tokens-dominate-cost/

https://github.com/allanninal/llm-api-fixes/tree/main/output-tokens-dominate-cost

---

54 **529 overloaded errors arrive in clusters and get dropped**

Three minutes on a Tuesday afternoon, and about fourteen hundred jobs are simply not in the output table. Nothing crashed. The worker's error counter shows a spike of something it logged as unexpected status, because the client special-cases 429 and 500 and lets everything else fall through to a generic failure path that drops the work. The status was 529, the platform was over capacity for four minutes, and the only trace left is that Anthropic did no work in those minutes while your client believed it was sending plenty.

`anthropic_overload_residual.py`

https://www.allanninal.dev/llm/overloaded-529-clusters/

https://github.com/allanninal/llm-api-fixes/tree/main/overloaded-529-clusters

---

55 **Parallel tool calls void the strict schema guarantee**

The schemas are strict, every one of them, because somebody read the Structured Outputs page properly and did the work. The parser has no try/except around it, deliberately: the arguments are guaranteed to conform, so a failure there should be loud. It has been loud four times in three months, always on a Tuesday afternoon, always with a stack trace that says a required field was missing, and always unreproducible from the same prompt. The four turns have one thing in common that nobody looked at: each of them called two tools instead of one.

`openai_parallel_strict_calls.py`

https://www.allanninal.dev/llm/parallel-tool-calls-with-strict-schema/

https://github.com/allanninal/llm-api-fixes/tree/main/parallel-tool-calls-with-strict-schema

---

56 **per-customer cost is unknowable because tenants share a key**

Finance asks what the largest account costs to serve. It is a reasonable question and somebody says it will take an afternoon, because the Usage API has a group_by=user_id and the application has been sending a user field on every request since the first week. The afternoon produces a table with eleven rows in it. Nine are engineers, two are service accounts, and not one of them is a customer.

`openai_tenant_attribution_audit.py`

https://www.allanninal.dev/llm/per-tenant-cost-attribution-impossible/

https://github.com/allanninal/llm-api-fixes/tree/main/per-tenant-cost-attribution-impossible

---

57 **previous_response_id 404s once the parent has aged out**

The support assistant remembers everything, which is the entire product. A customer opens a thread in March, adds to it in April, and comes back in June to ask about the thing they described the first time. This time the request 404s. Not the model, not the key, not the prompt: the id of the message before this one, which your code has been passing forward faithfully for three months. Server-side conversation state is a convenience with an expiry date on it, and nobody read the date.

`openai_response_chain_probe.py`

https://www.allanninal.dev/llm/previous-response-id-chain-broken/

https://github.com/allanninal/llm-api-fixes/tree/main/previous-response-id-chain-broken

---

58 **Priority Tier never covered the model you migrated to**

The commitment was signed two years ago, and the whole point of it was the 529s. It worked: the overload errors stopped, the on-call rotation got quiet, and everybody moved on. Then the platform team migrated the main assistant to a newer model, which was the correct thing to do on every axis they measured, and nothing in the migration checklist mentioned tiers because service_tier was already set to auto and had been for two years. The 529s came back four months later and nobody connected the two events, because there is no error, no warning and no line on the invoice that says the tier stopped applying.

`anthropic_priority_tier_coverage.py`

https://www.allanninal.dev/llm/priority-tier-model-unsupported/

https://github.com/allanninal/llm-api-fixes/tree/main/priority-tier-model-unsupported

---

59 **A model permission policy that has never excluded anything**

The postmortem is short and everybody already knows how it ends. A nightly classification job that labels support tickets was pointed at the most capable model in the catalogue, because that is what the developer had in their editor from the last thing they built, and it ran that way for five weeks. The interesting question is not why they chose it. It is the one somebody asks at the end of the meeting, almost as an aside: what would have stopped it? And the answer is nothing, in that project or any other, because model access is open by default and the control that closes it is per project and opt-in.

`openai_project_model_policy_audit.py`

https://www.allanninal.dev/llm/project-model-permissions-unrestricted/

https://github.com/allanninal/llm-api-fixes/tree/main/project-model-permissions-unrestricted

---

60 **A project or workspace ceiling set below the org limit**

The staging project was created for isolation, which is the advice everybody gives, and somebody set its rate limit low on the way past because staging does not need much. Eleven months later that project id is in the production deployment, because reusing it was one line and creating a new one was a ticket. Production now 429s at a fifth of the traffic the organization is entitled to, the organization dashboard shows plenty of headroom, and every other team is fine.

`rate_limit_below_org_audit.py`

https://www.allanninal.dev/llm/project-rate-limit-below-org/

https://github.com/allanninal/llm-api-fixes/tree/main/project-rate-limit-below-org

---

61 **Prompt sits under the model's cache minimum, so nothing caches**

The cache_control breakpoint has been in that request builder since March, and the Haiku route has never reported a cache read. Not a small number: zero, in every daily bucket, on a key whose Opus traffic caches beautifully off the same code path. Nothing errored, because nothing was wrong. The prefix is about fifteen hundred tokens. Opus 5 starts caching at five hundred and twelve. Haiku 4.5 does not start until four thousand and ninety six. The parameter was accepted and thrown away.

`anthropic_cache_floor_bracket.py`

https://www.allanninal.dev/llm/prompt-below-model-cache-minimum/

https://github.com/allanninal/llm-api-fixes/tree/main/prompt-below-model-cache-minimum

---

62 **Cache hits fall away exactly when the fleet scales out**

The cached share was fine in staging and fine in the first week of the rollout. Then autoscaling started doing its job, and the discount went the wrong way. At three in the morning the prompt caches at seventy per cent; at two in the afternoon, on the same template, the same model and the same code, it caches at sixteen. Everybody's first instinct is that something about the busy path is different. Nothing about the busy path is different. There are simply more machines in it, and none of them has seen this prefix before.

`openai_cache_key_routing_scatter.py`

https://www.allanninal.dev/llm/prompt-cache-key-not-set/

https://github.com/allanninal/llm-api-fixes/tree/main/prompt-cache-key-not-set

---

63 **Every scheduled run starts on a cache that was evicted**

The nightly enrichment job has run at 02:00 for a year and its prompt has not changed since June. Its cached share is zero. Not low, and not erratic — zero, on the first hour, every single night, while the two hours that follow it cache at seventy-five per cent off the same prefix. Nothing is misconfigured. The entry written at 02:00 last night was evicted somewhere around 02:40, and by the time the job came back twenty-three hours later there was nothing left to match.

`openai_cache_cold_after_idle.py`

https://www.allanninal.dev/llm/prompt-cache-retention-left-at-default/

https://github.com/allanninal/llm-api-fixes/tree/main/prompt-cache-retention-left-at-default

---

64 **prompt caching was never switched on anywhere**

The system prompt is four thousand tokens of instructions, a tool catalogue and two worked examples. It is identical on every call, and there are three hundred thousand calls a month. It has been reprocessed at the full input rate every single time, because prompt caching is opt-in and nobody opted in. There is no error to find, no warning header, and no line in the invoice that says what this cost. There is only a field in the usage report that has been zero since the day the integration shipped.

`anthropic_prompt_cache_off.py`

https://www.allanninal.dev/llm/prompt-caching-never-used/

https://github.com/allanninal/llm-api-fixes/tree/main/prompt-caching-never-used

---

65 **Prompts overflow the context window and 400 as too long**

The retrieval step got better last quarter, so it returns eight chunks instead of five. The agent loop got longer, because agents do. The tool list grew by four definitions nobody costed. None of those three changes touched the prompt template, and none of them was reviewed as a capacity change, and one afternoon the request that has worked for a year comes back 400 with prompt is too long.

`anthropic_context_preflight.py`

https://www.allanninal.dev/llm/prompt-too-long-context-overflow/

https://github.com/allanninal/llm-api-fixes/tree/main/prompt-too-long-context-overflow

---

66 **Prompts, Evals and Agent Builder close: export, not rewrite**

The deprecation notice reads like the others and gets triaged like the others: three surfaces, one date, put it in the sprint after next. What is different does not appear anywhere in the notice. The reusable prompt referenced as pmpt_a1b2 in four services is four characters of your source code and several hundred words of somebody else's, and those words are on OpenAI's side of the line. On 1 December the code still compiles, the deploy still succeeds, and the prompt is not in the repository because it never was.

`sunset_export_audit.py`

https://www.allanninal.dev/llm/prompts-evals-agentbuilder-sunset/

https://github.com/allanninal/llm-api-fixes/tree/main/prompts-evals-agentbuilder-sunset

---

67 **429 credit_balance_exhausted retried forever as a rate limit**

Traffic did not degrade, it stopped. Every request comes back 429, your retry wrapper does what it was built to do, and eight hours later it is still doing it. The status code says slow down. The code field inside the body says the account is out of money, and nothing in the SDK draws a line between the two — RateLimitError is raised for both.

`openai_quota_wall_audit.py`

https://www.allanninal.dev/llm/quota-exhausted-not-rate-limited/

https://github.com/allanninal/llm-api-fixes/tree/main/quota-exhausted-not-rate-limited

---

68 **429s are retried blindly without reading which limit hit**

The handler is four lines long and it has been in production for a year. Catch the 429, sleep, retry, give up after three. It works, in the sense that the process does not crash. It has also never once told anybody which of three separate limiters emptied, because the answer was in the response headers and the handler caught an exception class instead of reading a response.

`anthropic_limiter_identify.py`

https://www.allanninal.dev/llm/rate-limit-429-limiter-unidentified/

https://github.com/allanninal/llm-api-fixes/tree/main/rate-limit-429-limiter-unidentified

---

69 **x-ratelimit-remaining sits near zero before any 429**

Nothing has failed. There is no incident, no 429 in the logs, no page. There is a number that says you are running on four percent of your token budget at four in the afternoon, and it arrives attached to every successful response your application has ever received, and no code you own has ever looked at it.

`openai_rate_limit_headroom.py`

https://www.allanninal.dev/llm/rate-limit-headers-near-exhaustion/

https://github.com/allanninal/llm-api-fixes/tree/main/rate-limit-headers-near-exhaustion

---

70 **Requests billed, zero output tokens: max_tokens refused**

The model constant changed in a one-line pull request, because the old id has a shutdown date and somebody diaried it properly. The deploy went out on Thursday. Nothing paged: the endpoint returns a 500 to the user and the retry wrapper swallows it, and the error-rate dashboard is scoped to the gateway rather than to this worker. What the organization usage report shows for Friday is eleven thousand requests against the new model, no input tokens, and no output tokens at all.

`openai_zero_output_buckets.py`

https://www.allanninal.dev/llm/reasoning-model-rejects-max-tokens/

https://github.com/allanninal/llm-api-fixes/tree/main/reasoning-model-rejects-max-tokens

---

71 **reasoning tokens are billed as output but never returned**

Somebody changed one model constant. The answers coming back are the same length as before, the prompts are the same length as before, the request count is flat — and the line for that model on the cost report went up by a factor of four. The tokens you are paying for were generated, billed at the output rate, and then not returned to you.

`openai_reasoning_token_audit.py`

https://www.allanninal.dev/llm/reasoning-tokens-billed-invisibly/

https://github.com/allanninal/llm-api-fixes/tree/main/reasoning-tokens-billed-invisibly

---

72 **The model refused and the refusal field was never read**

The support summariser stopped producing summaries for a particular kind of ticket. Not all of them, and not with an error: the row was written, the summary column held an empty string, and the dashboard that counts summaries counted them. It took a week and a customer complaint before anyone read a stored response end to end, and there it was, in a content item nobody's code had ever touched: the model had declined, politely, and said why.

`openai_refusal_channel.py`

https://www.allanninal.dev/llm/refusal-field-ignored/

https://github.com/allanninal/llm-api-fixes/tree/main/refusal-field-ignored

---

73 **A 32 MB request is rejected with 413 before Anthropic sees it**

The PDF is 24 MB, which everybody agreed was fine, because the context window is 200,000 tokens and a 24 MB PDF is nothing like 200,000 tokens. The request comes back 413 anyway, with a body that does not look like Anthropic's usual error envelope, and nothing about it appears in the usage report. It never reached Anthropic. It was refused by a proxy in front of the API, for a reason that has nothing to do with tokens.

`anthropic_request_bytes.py`

https://www.allanninal.dev/llm/request-too-large-413/

https://github.com/allanninal/llm-api-fixes/tree/main/request-too-large-413

---

74 **Request count tripled while token volume stayed flat**

Latency got worse over about a fortnight, in the way that nobody can date precisely. Then 429s started arriving at a volume that used to be comfortable, and the obvious explanation was growth, except that the invoice barely moved. Two numbers sit in the same usage report and they have come apart: requests are up three times and tokens are up not at all. Two thirds of the extra calls landed in seventeen hours out of a hundred and sixty-eight, which is not what a new customer looks like.

`openai_retry_storm_shape.py`

https://www.allanninal.dev/llm/requests-diverge-from-token-volume/

https://github.com/allanninal/llm-api-fixes/tree/main/requests-diverge-from-token-volume

---

75 **a retired model id still sitting in the code**

A batch job that runs on the first of the month failed on every request with 404 and "type": "not_found_error", message The requested resource could not be found. The endpoint is right, the key works, the same key runs the rest of the application all day. The model id in that job's params block was retired months ago, and nothing else in the codebase names it, so nothing else broke and nobody knew.

`anthropic_model_ids_audit.py`

https://www.allanninal.dev/llm/retired-model-id-still-in-code/

https://github.com/allanninal/llm-api-fixes/tree/main/retired-model-id-still-in-code

---

76 **The gateway strips the header your backoff depends on**

The backoff was reviewed, it is textbook, and it reads retry-after before it sleeps. It has also never once seen that header, because the reverse proxy in front of the API forwards a response header allowlist that somebody wrote in 2023 and it contains content-type. So the handler falls through to its default of one second, retries into a bucket that has not refilled, and every retry pushes the reset further out while the incident channel fills up with people saying the backoff must be broken.

`retry_after_header_probe.py`

https://www.allanninal.dev/llm/retry-after-header-ignored/

https://github.com/allanninal/llm-api-fixes/tree/main/retry-after-header-ignored

---

77 **system_fingerprint moved and seed stopped reproducing**

The golden-file suite has been green for eight months. It pins seed, it pins temperature to zero, it asserts on the exact string the model returned the day the fixtures were recorded, and everybody agreed at the time that this was a reasonable way to test an LLM because it worked. This morning forty of them fail. The diffs are trivial — a comma, a reordered clause, one adjective — and nothing in the repository changed. The commit that broke them is not in your repository.

`openai_fingerprint_drift.py`

https://www.allanninal.dev/llm/seed-determinism-unreliable/

https://github.com/allanninal/llm-api-fixes/tree/main/seed-determinism-unreliable

---

78 **A service account key that has never been rotated**

This one is the reward for having done it right. Somebody read the guidance, created service accounts, moved production onto them and deleted the personal keys, and every audit since has come back clean because every audit since has been looking for personal keys. The service account was created in February two years ago. Its key was minted the same afternoon. There has never been a second key, so there has never been a moment when two keys were valid at once, so rotation has never been a deploy with a rollback: it has always been a hard cutover with an outage on the other side of it. Which is why it keeps being scheduled for next quarter.

`openai_key_rotation_clock.py`

https://www.allanninal.dev/llm/service-account-key-never-rotated/

https://github.com/allanninal/llm-api-fixes/tree/main/service-account-key-never-rotated

---

79 **The Videos API closes and no successor model is listed**

The migration ticket is written before anybody reads the table properly, because everyone has done this before: find the retired id, look up the successor, change the string, ship. So the ticket says "migrate off sora-2" and it is assigned and estimated. Then somebody opens the deprecations page to fill in the target and finds the replacement column is empty. Not to be announced, not see the migration guide. Empty, for all five ids, because the thing being removed is not a model. It is the feature.

`sora_shutdown_inventory.py`

https://www.allanninal.dev/llm/sora-videos-api-no-replacement/

https://github.com/allanninal/llm-api-fixes/tree/main/sora-videos-api-no-replacement

---

80 **spend jumped week over week and no release explains it**

The invoice is three times last month's and nothing shipped. There is no error to look up, no status code, no failed request — the API worked perfectly all month, which is the problem. Somewhere in the last six weeks a cron went from hourly to every five minutes, or a prompt template grew a retrieved document, or a customer onboarded, and the feedback loop between that change and the bill is measured in weeks. By the time the number arrives, nobody remembers the week it started in.

`llm_spend_week_over_week.py`

https://www.allanninal.dev/llm/spend-spike-week-over-week/

https://github.com/allanninal/llm-api-fixes/tree/main/spend-spike-week-over-week

---

81 **Every response is stored and no endpoint will list them**

The question arrives from legal, not from engineering, and it is one sentence long: what customer data are we currently holding on the model provider's side. Everybody assumes somebody knows. Nobody does. The application logs what it sent and what came back, the provider stores the same thing independently, and when you go looking for the endpoint that lists what is stored there so you can answer the sentence, there is not one. Not for responses, not for conversations. There is no query. There is only whatever ids you happened to write down.

`openai_stored_state_probe.py`

https://www.allanninal.dev/llm/stored-responses-accumulating/

https://github.com/allanninal/llm-api-fixes/tree/main/stored-responses-accumulating

---

82 **streamed responses report no usage and the dashboard undercounts**

The internal cost dashboard has been trusted for a year. It reads the usage object off every response, sums it per project, multiplies by the price card and draws a line. In January the line and the invoice were within a few percent of each other. They are not now, and the gap is a third. Nothing in the dashboard is broken, and nothing in the pipeline dropped a record: the chat endpoint was switched to streaming in March, and a streamed chunk carries usage: null.

`openai_streaming_usage_gap.py`

https://www.allanninal.dev/llm/streaming-usage-lost/

https://github.com/allanninal/llm-api-fixes/tree/main/streaming-usage-lost

---

83 **strict omitted, so the JSON schema is only a suggestion**

The validator threw about once every fifty calls, always in production, never in a test. Sometimes an extra key nobody had asked for; sometimes an optional field missing that the schema said was required; once a number arriving as a string. It read like flakiness, so it got a retry, and the retry mostly worked, which settled the matter for four months. The schema had been attached to every one of those calls. It had also never once been enforced, because somebody had taken the strict flag out eleven months earlier to make a 400 go away.

`openai_advisory_schema.py`

https://www.allanninal.dev/llm/strict-false-schema-silently-ignored/

https://github.com/allanninal/llm-api-fixes/tree/main/strict-false-schema-silently-ignored

---

84 **JSON cut off mid-object because the ceiling was reached**

The extraction pipeline had been green for six weeks and then started dropping about one document in forty into the dead-letter queue with a JSONDecodeError. The traceback pointed at a worker three hops from any HTTP client, so the first day went on the queue and the second on the worker. What it turned out to be was a 200. The model had followed the schema exactly, the way strict mode promises, and had been cut off halfway through the fourth line item of an unusually long invoice. Everything about that request succeeded, including the bill.

`openai_truncated_structured_output.py`

https://www.allanninal.dev/llm/structured-output-truncated-by-length/

https://github.com/allanninal/llm-api-fixes/tree/main/structured-output-truncated-by-length

---

85 **The same body counts 30% more tokens on the newer model**

The migration went fine. The evaluations were better, the latency was acceptable, nothing 500ed, and the rollout took an afternoon. Three weeks later the finance channel asks why input spend is up by a third on flat traffic, and somebody else opens a ticket about retrieval quality, and a third person notices that the conversation compactor now triggers two turns earlier than it used to. None of these is a bug. They are all the same fact arriving in three different rooms: the number your code uses to mean how big is this was measured against a model you no longer call.

`anthropic_tokenizer_delta.py`

https://www.allanninal.dev/llm/token-counts-reused-across-tokenizers/

https://github.com/allanninal/llm-api-fixes/tree/main/token-counts-reused-across-tokenizers

---

86 **Almost everyone in the organization holds the owner role**

Nobody decided this. Someone needed to create a project in week three and the fastest unblock was to make them an owner; someone needed to raise a rate limit in month five and the same thing happened; the contractor needed a key for a fortnight in March. Every one of those was the right call at the time and none of them was ever undone, because demotion is a visible act with a social cost and nothing in the platform ever asks. Two years later the roster has fourteen names on it and thirteen can change the billing settings.

`openai_owner_ratio_audit.py`

https://www.allanninal.dev/llm/too-many-organization-owners/

https://github.com/allanninal/llm-api-fixes/tree/main/too-many-organization-owners

---

87 **Tool-call arguments that parse and still break the schema**

The agent had been running for a month when a run started dying in the middle of a turn, roughly once in three hundred calls, always on the same tool. The traceback was a KeyError inside the handler, four frames below anything that knew about HTTP. The arguments string had parsed without complaint. It was well-formed JSON, it was even plausible JSON, and it described a call to a function whose signature had changed in a pull request the tool schema had not followed.

`openai_tool_call_arguments.py`

https://www.allanninal.dev/llm/tool-call-arguments-unparseable/

https://github.com/allanninal/llm-api-fixes/tree/main/tool-call-arguments-unparseable

---

88 **Tool shipped on every request and never once called**

The tool registry has twenty-six entries. Nineteen of them were written in the same fortnight by four people, and the descriptions read like function signatures because that is what they were copied from. Every one of them goes out on every request, because the registry is built once at start-up and handed to the client whole. Nothing is broken. The handlers for six of those tools have not been entered in production since the day they were merged, and the tools are still being paid for on every turn.

`openai_dead_tool_definitions.py`

https://www.allanninal.dev/llm/tool-defined-but-never-called/

https://github.com/allanninal/llm-api-fixes/tree/main/tool-defined-but-never-called

---

89 **Tool schemas are most of the input tokens on every call**

The agent has forty tools because forty things are worth doing, and each definition is a careful JSON schema with descriptions on every property because that is how you get the arguments right. A support turn is one sentence from a customer. Nobody has ever asked what the block in front of that sentence weighs, and the answer, when somebody finally counts it, is that the machinery is thirteen times the conversation on every single call.

`anthropic_tool_schema_overhead.py`

https://www.allanninal.dev/llm/tool-schemas-dominate-input-tokens/

https://github.com/allanninal/llm-api-fixes/tree/main/tool-schemas-dominate-input-tokens

---

90 **Nobody has ever read the key lifecycle audit log**

The control exists. It has existed since the organization was created, it has recorded every key minted and every key deleted and every member added since then, and it is complete, accurate and correctly timestamped. It has also never been read by anybody, because reading it requires standing up a job, and a log that is silent when everything is healthy gives nobody a reason to build one. There is an entry in there from a Tuesday in March: a key created at 02:14 UTC by an email address that is no longer on the roster. It has been sitting there for five months.

`llm_key_lifecycle_review.py`

https://www.allanninal.dev/llm/unreviewed-key-lifecycle-in-audit-log/

https://github.com/allanninal/llm-api-fixes/tree/main/unreviewed-key-lifecycle-in-audit-log

---

91 **The same key works on your laptop and 403s in production**

The feature works. It works on two laptops, it works in CI, it worked in the preview deployment, and it has never once failed in review. It is promoted to production on a Thursday and every request returns 403. Not a rate limit, not an expired key, not a model id: a flat refusal with a message about countries. Nobody changed the code. Nobody rotated the key. What changed is that the edge platform picked a point of presence closer to the user, and the request now leaves the internet somewhere the provider does not serve.

`llm_egress_region_probe.py`

https://www.allanninal.dev/llm/unsupported-country-region/

https://github.com/allanninal/llm-api-fixes/tree/main/unsupported-country-region

---

92 **US inference geo is billing every token at 1.1x**

Somebody in a procurement call last spring said that the data has to stay in the US, and somebody else, being helpful, went and set the workspace default that afternoon. Nobody wrote it down, because it took four seconds. The contract that prompted it was signed for one customer. The workspace serves all of them, and every token any of them has generated since has been billed at one and a tenth times the rate card.

`anthropic_inference_geo_premium_audit.py`

https://www.allanninal.dev/llm/us-inference-geo-premium-unnoticed/

https://github.com/allanninal/llm-api-fixes/tree/main/us-inference-geo-premium-unnoticed

---

93 **A vector store with expires_after deletes itself on a clock**

The retrieval demo was built in one good afternoon in March, shown twice, and then left alone while the team shipped something else. In May somebody asks to show it again, it comes up, and it answers every question out of the model's own head. The store id is unchanged and still in the config. The store still exists. Its status is expired, its file_counts are all zero, and the file objects it held were deleted on a schedule that was set at creation by a tool nobody remembers configuring.

`openai_vector_store_expiry_audit.py`

https://www.allanninal.dev/llm/vector-store-expired-or-expiring/

https://github.com/allanninal/llm-api-fixes/tree/main/vector-store-expired-or-expiring

---

94 **Files failed to index and file_search quietly returns less**

Somebody in support says the assistant does not know about the September pricing change, and you know for a fact that the September pricing memo was in the ingest. You check, and it was: the upload succeeded, the attach call returned 200, the ingest job logged 812 files indexed and exited zero. The store's status says completed. Every one of those things is true, and the memo is a scanned PDF with no text layer, so it is not in the index and never was.

`openai_vector_store_attach_failures.py`

https://www.allanninal.dev/llm/vector-store-file-attach-failed/

https://github.com/allanninal/llm-api-fixes/tree/main/vector-store-file-attach-failed

---

95 **Vector store bytes grow while nobody queries the index**

Somebody finally asks about the small line. It has been on the invoice for fourteen months, it has never been more than a couple of hundred dollars, and it is the only line that has gone up every single month regardless of what shipped. It is vector store storage. It is billed on bytes retained per hour rather than on anything anybody did, and a good deal of it is a corpus indexed for a demo in the spring of last year that has not been searched since the demo.

`openai_vector_store_storage_trend.py`

https://www.allanninal.dev/llm/vector-store-storage-cost-creeping/

https://github.com/allanninal/llm-api-fixes/tree/main/vector-store-storage-cost-creeping

---

96 **web search is billing $10 per 1,000 searches unnoticed**

The research agent was allowed to search the web in March, because an agent that cannot look anything up is a party trick. Nobody put a ceiling on how many times it could search in one turn, because in March it searched twice. It now runs on every support ticket, it searches until it is satisfied, and satisfied averages eleven. None of that is a token, so the token dashboard on the wall behind the standup has never once flickered.

`anthropic_web_search_spend_audit.py`

https://www.allanninal.dev/llm/web-search-spend-unnoticed/

https://github.com/allanninal/llm-api-fixes/tree/main/web-search-spend-unnoticed

---

97 **Zero data retention is claimed and no project resolves to it**

The security questionnaire came back with one line highlighted. Prompts and completions are not retained by the model provider. Somebody wrote that eighteen months ago and it was true of the arrangement negotiated at the time, and nobody has looked since, because there is nothing to look at: no header comes back on a completion saying what the retention mode was, no field in the response, no warning in the logs. Meanwhile four projects have been created since that contract was signed, and each one started life on whatever the organization default happened to be that morning.

`openai_data_retention_audit.py`

https://www.allanninal.dev/llm/zero-data-retention-not-configured/

https://github.com/allanninal/llm-api-fixes/tree/main/zero-data-retention-not-configured

---